

PROJECT REPORT

on

LIBRARY MANAGEMENT SYSTEM

Developed using Python and SQLite

Submitted in partial fulfilment of the requirements for the award of the degree of

Bachelor of Technology

Computer Science and Engineering (Artificial Intelligence and Machine Learning)

Submitted by

Deepika V

Register No: 312425148013

Roll No: 25AM1143

Department of Computer Science and Engineering (AIML)
St. Joseph's Institute of Technology

Academic Year 2026 - 2027

Table of Contents

1. Introduction	2
2. Abstract	3
3. Problem Statement	4
4. Methodology	5
5. System Architecture	6
6. Objectives	9
7. Future Enhancement	10
8. Code and Output	11
9. Conclusion	18
10. References	19

1. Introduction

A library is a collection of books and resources that must be organised so that readers can find and borrow them quickly. In many small and medium-sized institutions, records of books, members and borrowing are still maintained in paper registers or spreadsheets. These methods are slow, prone to human error, and make it difficult to know at any moment which books are available and which are overdue.

The **Library Management System (LMS)** developed in this project is a software application that automates these day-to-day tasks. It is written in **Python** and stores its data in a **SQLite** relational database. The system lets a librarian add books, register members, issue and return books, automatically calculate late-return fines, and generate reports on issued books and stock.

Python was chosen for its readability and rich standard library, and SQLite because it is a serverless, zero-configuration database that is bundled with Python through the `sqlite3` module. This keeps the system lightweight, portable and easy to deploy on any computer.

2. Abstract

This project presents the design and implementation of a console-based Library Management System that replaces manual record keeping with a reliable, database-backed application. The system is organised into three layers: a presentation layer (menu-driven interface), a business-logic layer (the *Library* class) and a data layer (SQLite database with three normalised tables: *books*, *members* and *transactions*).

The application supports adding and searching books by title, author or category; registering members; issuing books with a 14-day loan period; returning books with an automatic fine of Rs. 2 per late day; and producing issued-book and inventory reports. Input validation prevents issuing a book that does not exist or has no available copies, and prevents a return from being processed twice.

Testing with a scripted demonstration confirmed that stock counts update correctly after every issue and return, that fines are computed accurately, and that invalid operations are rejected with clear messages. The design is modular and can be extended with a graphical interface, authentication and email notifications.

Keywords: Library Management, Python, SQLite, Database, Fine Calculation, Inventory.

3. Problem Statement

Libraries that rely on manual registers or loosely organised spreadsheets face several recurring problems:

- **Slow searching** - locating a book by title, author or subject requires scanning records by hand.
- **Inaccurate stock** - the number of available copies is not updated reliably when books are issued or returned.
- **Manual fine calculation** - overdue fines are computed by hand, leading to errors and disputes.
- **Duplicate or lost records** - paper records can be damaged, misplaced or entered twice.
- **No quick reports** - finding which books are currently issued, and to whom, is time-consuming.

There is therefore a need for a simple, low-cost software system that stores library data centrally, enforces borrowing rules automatically, and provides instant search and reporting without requiring a dedicated database server.

4. Methodology

The project followed the **Waterfall** software development model, in which each phase is completed before the next begins. This suits a project with clearly defined requirements.

Phase	Activities
Requirement Analysis	Identified users (librarian), required functions (books, members, issue/return, fines, reports) and constraints (offline, no server).
Design	Designed the three-layer architecture, the entity-relationship model and the table schema.
Implementation	Coded the system in Python using the built-in sqlite3 module; one class encapsulates all database operations.
Testing	Executed a scripted scenario covering normal and error cases; verified stock counts and fine values.
Deployment & Maintenance	Delivered as a single Python file; the database file is created automatically on first run.

4.1 Tools and Technologies

Component	Technology
Programming language	Python 3.8 or later
Database	SQLite 3 (via sqlite3 module)
Interface	Console menu and Flask web interface
Editor / IDE	Any (VS Code, IDLE, PyCharm)
Operating system	Windows / Linux / macOS

4.2 Business Rules

- A book can be issued only if at least one copy is available.
- The loan period is 14 days from the issue date.
- A late return is charged a fine of Rs. 2 for every day after the due date.
- A transaction can be returned only once.

5. System Architecture

The system uses a layered architecture that separates user interaction, application logic and data storage. This separation makes the code easier to understand, test and extend.

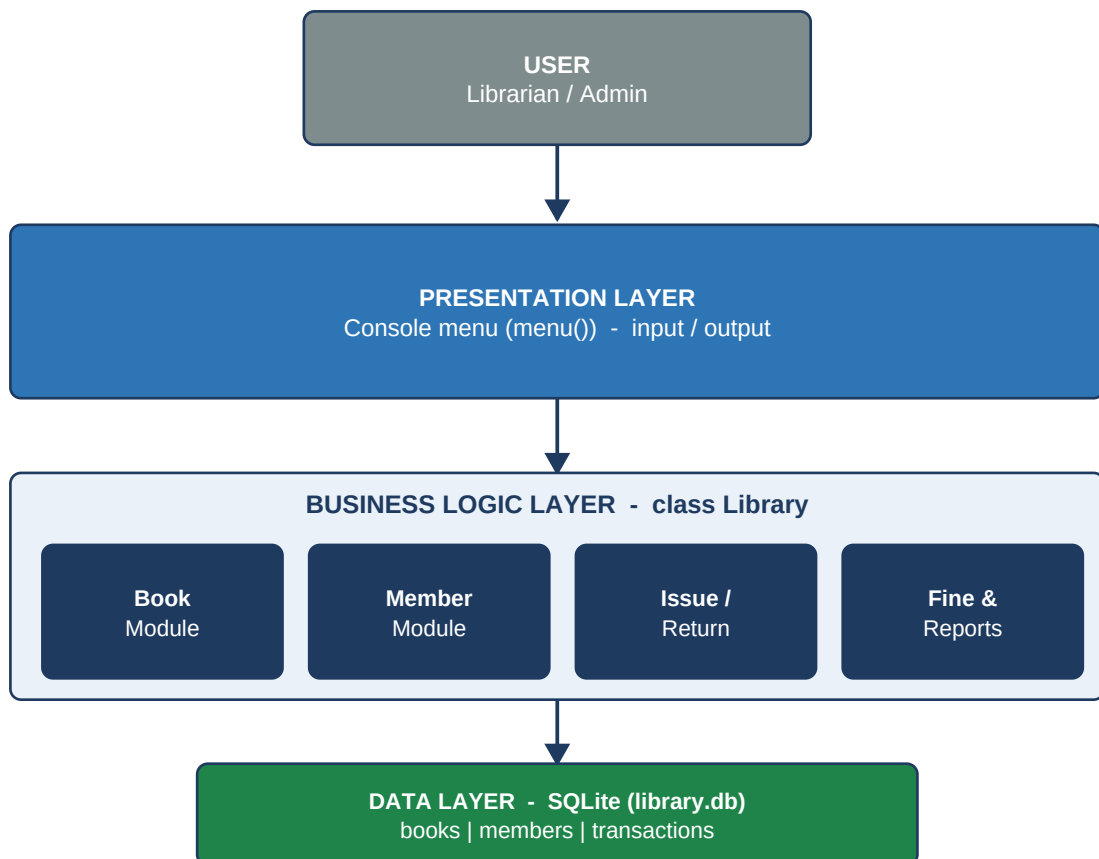


Figure 5.1: Layered architecture of the Library Management System

5.1 Database Design

The database is normalised into three tables. Each transaction links one book to one member, so a book can appear in many transactions (one-to-many) and a member can borrow many books over time.

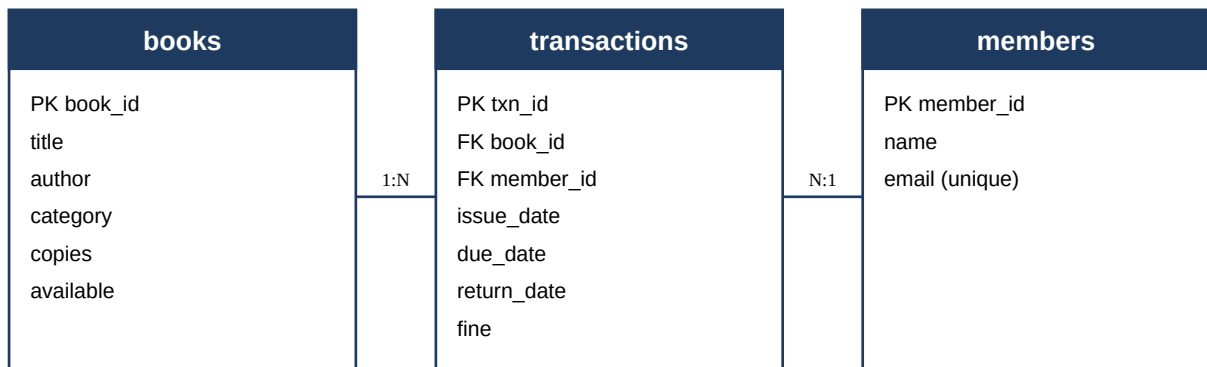


Figure 5.2: Entity-relationship diagram

5.2 Table Structure

Table	Column	Type	Description
books	book_id	INTEGER PK	Unique book identifier (auto-increment)
	title, author	TEXT	Book details
	category	TEXT	Subject / genre
	copies	INTEGER	Total copies owned
	available	INTEGER	Copies currently on the shelf
members	member_id	INTEGER PK	Unique member identifier
	name	TEXT	Member name
	email	TEXT UNIQUE	Contact e-mail
transactions	txn_id	INTEGER PK	Unique transaction identifier
	book_id, member_id	INTEGER FK	References books and members
	issue_date, due_date	TEXT	ISO dates (YYYY-MM-DD)
	return_date	TEXT	NULL while the book is still issued
	fine	INTEGER	Fine charged in rupees

5.3 Module Description

- **Book module** - add_book(), search_books().
- **Member module** - add_member().
- **Circulation module** - issue_book(), return_book() with stock update and fine calculation.
- **Report module** - issued_report(), inventory_report(), stats(), history().

- **Web interface** - app.py (Flask) provides a dashboard and pages for books, members and issue / return on top of the same Library class.

6. Objectives

The main objectives of this project are:

- To replace manual register-based record keeping with a computerised system.
- To store book, member and transaction data permanently in a relational database.
- To provide fast searching of books by title, author or category.
- To automate issuing and returning of books while keeping stock counts accurate.
- To calculate overdue fines automatically and without error.
- To generate reports of currently issued books and current inventory.
- To build a lightweight, portable system that needs no separate database server.

7. Future Enhancement

The present system meets its core objectives, and its modular design makes the following improvements straightforward:

- **Richer web interface** - a basic Flask interface is already included; it can be extended with pagination, editing and deleting records.
- **User authentication** with separate roles for administrators, librarians and students.
- **E-mail / SMS reminders** before the due date and for overdue books.
- **Barcode or QR-code scanning** for quick issue and return.
- **Reservation system** allowing members to reserve a book that is currently unavailable.
- **Advanced reports and charts** such as most-borrowed books and monthly circulation.
- **Cloud database** (MySQL / PostgreSQL) to support multiple branches.
- **Online payment** of fines and a member self-service portal.

8. Code and Output

8.1 Source Code (library_system.py)

```
"""Library Management System - Python + SQLite
Features: add/search books, register members, issue/return with fine calculation, reports.
"""
import sqlite3
from datetime import date, timedelta

DB_FILE = "library.db"
LOAN_DAYS = 14
FINE_PER_DAY = 2 # Rs. per day late

class Library:
    def __init__(self, db_file=DB_FILE):
        self.conn = sqlite3.connect(db_file)
        self.conn.row_factory = sqlite3.Row
        self._create_tables()

    def close(self):
        self.conn.close()

    def _create_tables(self):
        self.conn.executescript("""
        CREATE TABLE IF NOT EXISTS books(
            book_id INTEGER PRIMARY KEY AUTOINCREMENT,
            title TEXT NOT NULL, author TEXT NOT NULL,
            category TEXT, copies INTEGER NOT NULL DEFAULT 1,
            available INTEGER NOT NULL DEFAULT 1);
        CREATE TABLE IF NOT EXISTS members(
            member_id INTEGER PRIMARY KEY AUTOINCREMENT,
            name TEXT NOT NULL, email TEXT UNIQUE);
        CREATE TABLE IF NOT EXISTS transactions(
            txn_id INTEGER PRIMARY KEY AUTOINCREMENT,
            book_id INTEGER REFERENCES books(book_id),
            member_id INTEGER REFERENCES members(member_id),
            issue_date TEXT NOT NULL, due_date TEXT NOT NULL,
            return_date TEXT, fine INTEGER DEFAULT 0);
        """)
        self.conn.commit()

    # ---- Books ----
    def add_book(self, title, author, category, copies=1):
        cur = self.conn.execute(
            "INSERT INTO books(title,author,category,copies,available) VALUES(?,?,?,?,?)",
            (title, author, category, copies, copies))
        self.conn.commit()
        return cur.lastrowid

    def search_books(self, keyword):
        like = f"%{keyword}%"
        return self.conn.execute(
            "SELECT * FROM books WHERE title LIKE ? OR author LIKE ? OR category LIKE ?",
            (like, like, like)).fetchall()

    # ---- Members ----
    def add_member(self, name, email):
        cur = self.conn.execute("INSERT INTO members(name,email) VALUES(?,?)", (name, email))
        self.conn.commit()
        return cur.lastrowid

    # ---- Issue / Return ----
    def issue_book(self, book_id, member_id, on=None):
        on = on or date.today()
        book = self.conn.execute("SELECT * FROM books WHERE book_id=?", (book_id,)).fetchone()
        if book is None:
            return "Error: book not found"
        if book["available"] < 1:
            return "Error: no copies available"
        due = on + timedelta(days=LOAN_DAYS)
        self.conn.execute(
```

```
        "INSERT INTO transactions(book_id,member_id,issue_date,due_date) VALUES(?,?,?,?)",
        (book_id, member_id, on.isoformat(), due.isoformat()))
    self.conn.execute("UPDATE books SET available=available-1 WHERE book_id=?", (book_id,))
    self.conn.commit()
    return f"Issued '{book['title']}' - due on {due.isoformat()}"

def return_book(self, txn_id, on=None):
    on = on or date.today()
    t = self.conn.execute("SELECT * FROM transactions WHERE txn_id=? AND return_date IS NULL",
                          (txn_id,)).fetchone()

    if t is None:
        return "Error: active transaction not found"
    late = (on - date.fromisoformat(t["due_date"])).days
    fine = max(0, late) * FINE_PER_DAY
    self.conn.execute("UPDATE transactions SET return_date=?, fine=? WHERE txn_id=?",
                     (on.isoformat(), fine, txn_id))
    self.conn.execute("UPDATE books SET available=available+1 WHERE book_id=?", (t["book_id"],))
    self.conn.commit()
    return f"Returned. Late by {max(0, late)} day(s). Fine: Rs.{fine}"

# ---- Listing helpers (used by the web interface) ----
def list_books(self):
    return self.conn.execute("SELECT * FROM books ORDER BY title").fetchall()

def list_members(self):
    return self.conn.execute("SELECT * FROM members ORDER BY name").fetchall()

def history(self, limit=15):
    return self.conn.execute("""
        SELECT t.txn_id, b.title, m.name, t.issue_date, t.return_date, t.fine
        FROM transactions t JOIN books b USING(book_id) JOIN members m USING(member_id)
        WHERE t.return_date IS NOT NULL ORDER BY t.return_date DESC, t.txn_id DESC LIMIT ?""",
        (limit,)).fetchall()

def stats(self, today=None):
    today = (today or date.today()).isoformat()
    q = lambda sql, *a: self.conn.execute(sql, a).fetchone()[0]
    return {
        "titles": q("SELECT COUNT(*) FROM books"),
        "copies": q("SELECT COALESCE(SUM(copies),0) FROM books"),
        "available": q("SELECT COALESCE(SUM(available),0) FROM books"),
        "issued": q("SELECT COUNT(*) FROM transactions WHERE return_date IS NULL"),
        "overdue": q("SELECT COUNT(*) FROM transactions WHERE return_date IS NULL AND due_date < ?", today),
        "members": q("SELECT COUNT(*) FROM members"),
        "fines": q("SELECT COALESCE(SUM(fine),0) FROM transactions"),
    }

# ---- Reports ----
def issued_report(self):
    return self.conn.execute("""
        SELECT t.txn_id, b.title, m.name, t.issue_date, t.due_date
        FROM transactions t JOIN books b USING(book_id) JOIN members m USING(member_id)
        WHERE t.return_date IS NULL ORDER BY t.due_date""").fetchall()

def inventory_report(self):
    return self.conn.execute("SELECT book_id,title,copies,available FROM books").fetchall()

def menu():
    lib = Library()
    options = ""
    ===== LIBRARY MANAGEMENT SYSTEM =====
    1. Add Book      2. Search Books    3. Add Member
    4. Issue Book    5. Return Book     6. Issued Books Report
    7. Inventory     0. Exit""
    while True:
        print(options)
        ch = input("Enter choice: ").strip()
        if ch == "1":
            print("Book ID:", lib.add_book(input("Title: "), input("Author: "),
                                           input("Category: "), int(input("Copies: "))))
        elif ch == "2":
            for b in lib.search_books(input("Keyword: ")):
                print(b["book_id"], b["title"], "-", b["author"], f"({b['available']}/{b['copies']})")
        elif ch == "3":
            print("Member ID:", lib.add_member(input("Name: "), input("Email: ")))
```

```
        elif ch == "4":
            print(lib.issue_book(int(input("Book ID: ")), int(input("Member ID: "))))
        elif ch == "5":
            print(lib.return_book(int(input("Transaction ID: "))))
        elif ch == "6":
            for r in lib.issued_report():
                print(tuple(r))
        elif ch == "7":
            for r in lib.inventory_report():
                print(tuple(r))
        elif ch == "0":
            break
        else:
            print("Invalid choice")

if __name__ == "__main__":
    menu()
```

8.2 Demonstration Script (demo.py)

The following script exercises every feature of the system with sample data, including error cases. Dates are set explicitly so that the late-fine calculation can be verified.

```
"""Non-interactive demo that exercises every feature and prints the output."""
import os
from datetime import date, timedelta
from library_system import Library

if os.path.exists("demo.db"): os.remove("demo.db")
lib = Library("demo.db")
today = date(2026, 10, 1)

print(">>> Adding books")
for t, a, c, n in [("Python Crash Course", "Eric Matthes", "Programming", 3),
                  ("Database System Concepts", "Silberschatz", "Databases", 2),
                  ("Wings of Fire", "A.P.J. Abdul Kalam", "Biography", 1),
                  ("Clean Code", "Robert C. Martin", "Programming", 2)]:
    print(f"Book ID {lib.add_book(t, a, c, n)} -> {t}")

print("\n>>> Registering members")
print("Member ID", lib.add_member("Arun Kumar", "arun@example.com"))
print("Member ID", lib.add_member("Priya Raman", "priya@example.com"))

print("\n>>> Searching for 'Programming'")
for b in lib.search_books("Programming"):
    print(b["book_id"], b["title"], "-", b["author"], f"({b['available']}/{b['copies']} available)")

print("\n>>> Issuing books")
print(lib.issue_book(1, 1, on=today - timedelta(days=20)))
print(lib.issue_book(3, 2, on=today - timedelta(days=5)))
print(lib.issue_book(3, 1, on=today))
print(lib.issue_book(99, 1, on=today))

print("\n>>> Issued books report")
for r in lib.issued_report():
    print(tuple(r))

print("\n>>> Returning books")
print(lib.return_book(1, on=today))
print(lib.return_book(1, on=today))

print("\n>>> Inventory")
for r in lib.inventory_report():
    print(tuple(r))
```

8.3 Program Output

```
>>> Adding books
Book ID 1 -> Python Crash Course
Book ID 2 -> Database System Concepts
Book ID 3 -> Wings of Fire
Book ID 4 -> Clean Code

>>> Registering members
Member ID 1
Member ID 2

>>> Searching for 'Programming'
1 Python Crash Course - Eric Matthes (3/3 available)
4 Clean Code - Robert C. Martin (2/2 available)

>>> Issuing books
Issued 'Python Crash Course' - due on 2026-09-25
Issued 'Wings of Fire' - due on 2026-10-10
Error: no copies available
Error: book not found

>>> Issued books report
(1, 'Python Crash Course', 'Arun Kumar', '2026-09-11', '2026-09-25')
(2, 'Wings of Fire', 'Priya Raman', '2026-09-26', '2026-10-10')

>>> Returning books
Returned. Late by 6 day(s). Fine: Rs.12
Error: active transaction not found

>>> Inventory
(1, 'Python Crash Course', 3, 3)
(2, 'Database System Concepts', 2, 2)
(3, 'Wings of Fire', 1, 0)
(4, 'Clean Code', 2, 2)
```

8.4 Output Analysis

Test	Expected result	Observed
Issue available book	Issued, due date = issue + 14 days	Passed (due 2026-09-25)
Issue last copy, then again	Second request rejected	Passed (no copies available)
Issue unknown book ID	Error message	Passed (book not found)
Return 6 days late	Fine = 6 x Rs. 2 = Rs. 12	Passed (Rs. 12)
Return same transaction twice	Second return rejected	Passed (not found)
Stock after operations	Wings of Fire available = 0	Passed

8.5 Web Interface Screenshots

The same Library class also drives a Flask web application (app.py). The screenshots below show the dashboard and the Issue / Return page, where an overdue book is flagged with its fine.

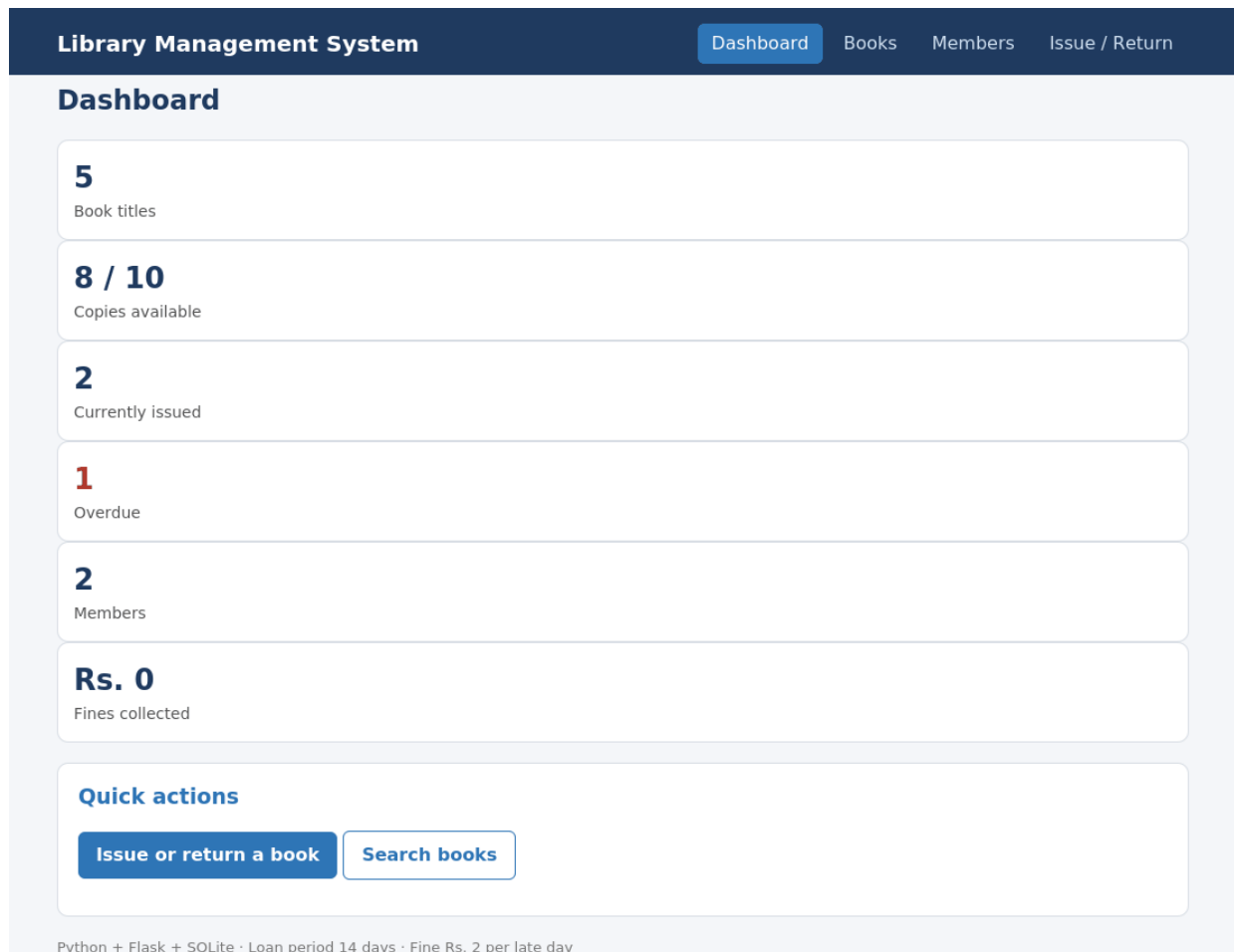


Figure 8.1: Dashboard

Library Management System

[Dashboard](#)[Books](#)[Members](#)[Issue / Return](#)

Issue / Return

Issue a book

Book

Member

Issue date (optional - back-date to test fines)

Issue

Select a book

Select a member

Currently issued

Txn	Book	Member	Issued	Due	Status	
1	Python Crash Course	Arun Kumar	2026-09-13	2026-09-27	6 day(s) late - fine Rs. 12	Return
2	Wings of Fire	Priya Raman	2026-09-28	2026-10-12	On time	Return

Recently returned

Txn	Book	Member	Issued	Returned	Fine
Nothing returned yet.					

Python + Flask + SQLite · Loan period 14 days · Fine Rs. 2 per late day

Figure 8.2: Issue / Return page showing a late book and its fine

9. Conclusion

The Library Management System successfully automates the core activities of a library: cataloguing books, registering members, issuing and returning books, calculating fines and producing reports. By using SQLite, the system keeps all data in a single portable file while still benefiting from the integrity and query power of a relational database.

Testing showed that the system enforces its business rules correctly, keeps stock counts consistent, and handles invalid inputs gracefully. The layered, modular design means that new features such as a graphical interface or user authentication can be added without rewriting the existing logic.

In summary, the project meets all of its stated objectives and provides a practical foundation for a more complete library automation solution.

10. References

- [1] Python Software Foundation, *Python 3 Documentation*. <https://docs.python.org/3/>
- [2] Python Software Foundation, *sqlite3 - DB-API 2.0 interface for SQLite databases*.
<https://docs.python.org/3/library/sqlite3.html>
- [3] SQLite Consortium, *SQLite Documentation*. <https://www.sqlite.org/docs.html>
- [4] A. Silberschatz, H. F. Korth, S. Sudarshan, *Database System Concepts*, McGraw-Hill Education.
- [5] R. Elmasri, S. B. Navathe, *Fundamentals of Database Systems*, Pearson.
- [6] I. Sommerville, *Software Engineering*, Pearson.
- [7] E. Matthes, *Python Crash Course*, No Starch Press.